

שנקר ביי"ס גבוה להנדסה ולעישוב
החוג להנדסת תוכנה
קורס 'רשתות תקשורת' (קורס מספר 3503836)
תשפ"ו – סמסטר ב'
מרצה: יצחק נודלר

תרגיל בית מס' 3 – פתרון

שאלה מס' 1

א. כיצד TCP מייצג פנימית, במימוש בתוכנה, אובייקט מסוג סוקט?

ב. האם הייצוג הפנימי של אובייקט מסוג סוקט על ידי TCP הממומש ב-kernel של Windows 11 זהה בהכרח לייצוג הפנימי של אובייקט מסוג סוקט על ידי TCP הממומש ב-kernel של Linux?

במערכות הפעלה מודרניות, ה-Socket אינו רק אובייקט תוכנה פשוט, אלא מנגנון מורכב המגשר בין העולם של ה-User Space (האפליקציה) לבין ה-Kernel Space (עולם התקשורת).

להלן תשובות לשאלות הנ"ל מנקודת מבט של הנדסת מערכות הפעלה:

א. כיצד TCP מייצג פנימית אובייקט מסוג סוקט?

במימוש ה-Kernel, ה-Socket מיוצג לרוב כמבנה נתונים (Struct) המכיל את כל ה"סטטייט" (State) הנדרש לניהול חיבור ה-TCP. ניתן לחלק את הייצוג הזה למספר שכבות לוגיות:

1. מבנה ה-Control Block (TCB - Transmission Control Block):

זהו הלב של הסוקט. הוא מכיל את המידע שמוגדר ב-RFC 793 (ועדכוניו), כגון:

- **מכונת מצבים (Finite State Machine):** משתנה המציין את מצב החיבור הנוכחי (LISTEN, SYN_SENT, ESTABLISHED, TIME_WAIT וכו').
- **מספרי רצף:** המשתנים הנוכחיים עבור Send Sequence Numbers ו-Receive Sequence Numbers.
- **ערכי חלונות:** גודל חלון הקבלה והשידור המעודכנים.

2. ניהול חוצצים (Buffers):

הסוקט מחזיק מצביעים (Pointers) ל-Send Buffer ו-Receive Buffer. אלו אינם רק מערכי זיכרון פשוטים, אלא לעיתים קרובות מדובר בשרשראות של מבני נתונים (כמו sk_buff ב-Linux), המאפשרים ל-Kernel לנהל את הנתונים בצורה יעילה ללא העתקות זיכרון מיותרות (Zero-copy).

3. תורים (Queues):

הסוקט מנהל תור של חבילות הממתינות לשידור, תור של חבילות שהגיעו אך טרם עובדו, ותור של חיבורים (במצב LISTEN) שממתינים ל-accept().

4. ממשק למערכת ההפעלה (File Descriptor / Handle):

במערכות דמויות Unix, הסוקט ממופה ל- File Descriptor. ב- Windows, הוא ממופה ל- Handle. ה- Kernel מקשר את ה- Handle הזה למבנה הנתונים הפנימי של ה- Socket.

ב. האם הייצוג ב- Windows 11 זהה לזה שב- Linux?

התשובה היא חד-משמעית לא. הייצוג הפנימי שונה בתכלית, מסיבות ארכיטקטוניות והיסטוריות:

1. הבדלים בארכיטקטורת ה- Kernel

- **Linux (Monolithic Kernel):** ה- Socket ב- Linux מבוסס על מבנה נתונים שנקרא struct sock. הוא עוצב להיות רזה ומהיר ככל האפשר, עם אינטגרציה עמוקה מאוד לתוך ה- Networking Stack המודולרי של הלינוקס. הטיפול במידע מתבצע דרך (sk_buff Socket) (Buffer), שהיא מבנה הנתונים הסטנדרטי להעברת חבילות בין שכבות ה- Kernel השונות.
- **Windows (Hybrid/NT Kernel):** ה- Windows Networking Stack מבוסס על ארכיטקטורת ה- TDI (Transport Driver Interface) הישנה, ובחמשך על ה- WFP (Windows Filtering Platform) וה- Winsock Kernel (WSK). אובייקט ה- Socket ב- Windows הוא חלק ממערכת אובייקטים של ה- Kernel (Executive Objects). הוא מושפע מאוד מהצורך לתמוך ב- I/O Completion Ports (IOCP), שהוא מנגנון אסינכרוני ייחודי ל- Windows שאינו קיים באותה צורה בלינוקס.

2. ניהול הזיכרון והתזמון

- ב- Linux, ניהול הסוקטים מותאם מאוד ל- Scheduler של לינוקס ולמבני ה- Locking הפנימיים שלה (כגון spinlocks ו- RCU לקריאות מהירות).
- ב- Windows, האובייקט צריך להשתלב עם ה- I/O Manager של ה- Kernel, מה שמחייב שדות נוספים במבנה הנתונים עבור "Packet Information" ומיטני ל- IRP (I/O Request Packets).

3. תאימות לממשקי תכנות

למרות שהן ה- Linux API (Berkeley Sockets) והן ה- Windows API (Winsock) שואפים לתאימות ברמת האפליקציה, ה"מאחורי הקלעים" שונים לחלוטין כדי לשרת את אופטימיזציות הביצועים הייחודית לכל מערכת.

לסיכום: בעוד ששתי המערכות צריכות ליישם את אותו פרוטוקול (TCP) לפי ה- RFCs, הן מממשות את "האובייקט" ככלי עבודה שונה לחלוטין כדי להתאים למבנה ה- Kernel שלהן ולמנגנוני ה- I/O המובנים בהן.

שאלה מס' 2

א. האם המימוש של TCP ב- kernel של Windows 11 חייב לשמור את כל 7 המאפיינים של פרוטוקול ה- TCP אותם תיארו בהרצאה?

ב. האם המימוש של TCP ב- kernel של Linux חייב לשמור את כל 7 המאפיינים של פרוטוקול ה- TCP אותם תיארו בהרצאה?
את כל 7 המאפיינים?

התשובה הקצרה והחד-משמעית לשתי השאלות היא כן.

כדי להבין מדוע זה המצב, נדרש להבחין בין הפרוטוקול (הסטנדרט) לבין המימוש (הקוד בתוך ה-Kernel).

א. המימוש ב-Windows 11

כן, המימוש חייב לשמור על כל מאפייני הפרוטוקול.

- **הסבר:** TCP מוגדר על ידי ה-IETF (Internet Engineering Task Force) בסדרת מסמכי RFC (הבסיסי שבהם הוא RFC 9293, המעדכן את RFC 793 ההיסטורי). כדי שמחשב שמריץ Windows 11 יוכל לתקשר עם מחשב שמריץ Linux, macOS, או כל מכשיר קצה אחר (כמו נתב או שרת), הוא חייב לדבוק בסטנדרט.
- אם Windows 11 היה מוותר על אחד מהמאפיינים (למשל, מנגנון ה-Flow Control או מנגנון ה-Retransmission), הוא היה מפסיק להיות "TCP". המשמעות המעשית תהיה קריסה של התקשורת מול כל יתר הרשת העולמית.
- **ה-Kernel של Windows משתמש ב-`tcpip.sys` (הנהג האחראי על ה-Stack), והוא מחויב ליישם את כל הלוגיקה של ניהול החיבור, אימות הנתונים, בקרת העומסים והסדר כדי להבטיח את ה-`interoperability` (יכולת פעולה הדדית) הקריטית לאינטרנט.**

ב. המימוש ב-Linux

כן, גם כאן המימוש חייב לשמור על כל המאפיינים.

- **הסבר:** תעשיית התוכנה, ובמיוחד מערכות ההפעלה (Unix-like ו-Windows כאחד), מבוססות על הסכמה גלובלית על פרוטוקולים. לינוקס, בהיותה הבסיס לרוב השרתים בעולם, מקפידה על הסטנדרטים הללו בצורה מחמירה מאוד.
- המימוש ב-Kernel של לינוקס **(הנמצא בדרך כלל בתיקית `net/ipv4/tcp_*.c` בקוד המקור)** מתוכנן במדויק כדי להגיב לחבילות TCP לפי הכללים המקובלים. אם לינוקס לא הייתה שומרת על מאפיין כמו "Ordered Delivery" (הבטחת סדר הגעת הנתונים), אפליקציות שרצות על לינוקס לא היו מצליחות להציג נתונים תקינים למשתמש.

הבהרה חשובה: סטנדרט מול אופטימיזציה

למרות שהם חייבים לשמור על המאפיינים (למשל: הבטחת אמינות, בקרת זרימה, סדר הגעה), **ישנם הבדלים באופן שבו כל Kernel מממש אותם "מתחת למכסה המנוע":**

1. **אלגוריתמי בקרת עומסים (Congestion Control):** הסטנדרט מחייב בקרת עומסים, אך הוא מאפשר גמישות בבחירת האלגוריתם. Linux עשויה להשתמש ב-BBR (פיתוח של גוגל) כברירת מחדל, בעוד Windows עשויה להשתמש ב-CUBIC או ב-Compound TCP. המאפיין (בקרת עומסים) קיים בשניהם, אך האסטרטגיה המתמטית שונה.
2. **אופטימיזציות מיוחדות:** כל Kernel יכול להוסיף "תוספות" (Extensions) לסטנדרט (כמו TCP Fast Open), בתנאי שהם אינם שוברים את התאימות לאחור עם צדדים שאינם תומכים בהם.

לסיכום: המאפיינים שתיארת בהרצאה הם ה-Contract (החוזה) הפרוטוקולי. ללא שמירה עליהם, התקשורת פשוט לא תתרחש. לכן, אין ל-Windows 11 או ל-Linux הפריבילגיה "לוותר" על אף אחד מהם.

שאלה מס' 3

האם הקריאה ל- `connect()` יכולה להיכשל?
כלומר להחזיר ערך חזרה שערכו 1-?
נמקו!

וודאי שהיא יכולה להיכשל.
לדוגמא, אם מחשב השרת הוא `down`.
או, דוגמא אחרת, צד השרת של האפליקציה הרשתית כלל לא רץ
ובאופן כללי: בכל מקרה ש- `TCP` בצד הלקוח לא יקבל מ- `TCP` בצד השרת, `SYN-ACK segment`
'בעקבות' שליחת ה- `SYN segment`

שאלה מס' 4

סטודנט כתב תוכנת לקוח השולחת מחרוזות טקסט בודדות לשרת `TCP` באמצעות קריאות `send()` עוקבות.
הסטודנט מניח כי עבור כל פעולת כתיבה בצד הלקוח, תתבצע פעולת קריאה `recv()` תואמת בצד השרת שתקבל בדיוק את אותה מחרוזת.
הסבירו מדוע הנחה זו שגויה ארכיטקטונית, מה קורה בפועל ב- `Kernel Buffers` של שכבת התעבורה, והציגו שתי שיטות הנדסיות מקובלות בשכבת האפליקציה (`Application-Level Framing`) לפתרון הבעיה.

אובדן גבולות ההודעה בממשק הזרם (`Byte-Stream`)
הפרוטוקול `TCP` הוא פרוטוקול המכוון לזרם בייטים (`Byte-Stream Oriented`) ואינו מודע או שומר על גבולות הודעה (`Message Boundaries`). מבחינת הגרעין (`Kernel`), המידע הנשלח הוא רצף אחד ורציף של אוקטטים (`Octets`).
כאשר הלקוח קורא ל- `write()`, המידע מועתק ל- `Socket Send Buffer` של מערכת ההפעלה. הגרעין רשאי לאחד מספר כתיבות קטנות לסגמנט (`Segment`) יחיד כדי למקסם את יעילות השרת (למשל **באמצעות אלגוריתם Nagle**), או לחלופין, לפצל כתיבה יחידה למספר סגמנטים בשל מגבלות ה- `MSS` (`Maximum Segment Size`) או אילוצי חלון הגודש (`cwnd`). בצד המקבל, המידע נצבר ב- `Receive Buffer` (וקריאת `read`) של האפליקציה שולפת את מה שזמין בבאפר באותו רגע – דבר שעלול לגרום לתופעות של איחוד הודעות (`Coalescing`) או קיטוע הודעות (`Fragmentation`).

- כדי לשחזר את ההודעות המקוריות בשכבת האפליקציה, יש ליישם תיחום מפורש (`Framing`):
- 1. `Length-Prefixed Framing`:** השולח מצרף לראש כל הודעה שדה בגודל קבוע (למשל, 4 בייטים) המציין את אורך הפיילוד (`Payload`) בבייטים. המקבל קורא ראשית את האורך, ואז מבצע קריאות בלולאה עד להשלמת מספר הבייטים המדויק.
 - 2. `Delimiter-Based Framing`:** שימוש בתו מוסכם או ברצף תווים מיוחד (כמו `rlal` בפרוטוקול `HTTP/1.1` או `01`) המציין את סוף ההודעה. המקבל סורק את זרם הבייטים ומפצל את ההודעות בהתאם לעוגן זה.

שאלה מס' 5

במהלך הרצאה, טען סטודנט כי המונחים "פורט" ו"סוקט" הם מילים נרדפות לאותו מושג הנדסי.

הפריכו טענה זו באופן חד-משמעי.

בתשובתכם, הגדירו במדויק כל מושג והסבירו כיצד שרת Web יחיד יכול לשרת אלפי לקוחות סימולטנית דרך פורט מקומי יחיד (למשל פורט 80).

אבסטרקט הסוקט לעומת מזהה הפורט בתכנות מערכות

הטענה שגויה מיסודה ומייצגת ערבוב מושגי חמור בין מזהה פרוטוקול לבין אבסטרקט תוכנה של מערכת ההפעלה:

- **Port:** הוא מזהה נומרי של שכבת התעבורה (16-bit). הוא מהווה רכיב בתוך שיטת המיפון של הרשת ומשמש ככתובת לוגית לנקודת גישה.
- **Socket:** הוא אבסטרקציה של מערכת ההפעלה (OS Endpoint Object) המשמשת לפעולות קלט/פלט של הרשת. במערכות הפעלה מודרניות (כמו לינוקס), סוקט מיוצג כמשאב מערכת ומנוהל באמצעות File Descriptor (FD) בתוך ה-Descriptor Table של התהליך.

מנגנון ה-TCP מזהה חיבורים [=connections] באופן חד-ערכי על בסיס **Tuple-4**:

Source IP

Source Port

Destination IP

Destination Port

כאשר שרת אינטרנט מאזין בפורט 80, יש לו סוקט האזנה יחיד (Listening Socket). כאשר לקוח א' מתחבר, הגרעין משלים את ה-Handshake ומייצר סוקט חיבור חדש (Connected Socket) המקבל File Descriptor נפרד, אך קשור לאותו פורט מקומי 80. אם לקוח ב' יתחבר ממחשב אחר (IP אחר) או מפורט מרוחק אחר, הגרעין ייצר עבורו סוקט חיבור נוסף (FD נפרד). כיוון שה-Tuple-4 של שני הלקוחות שונה, הגרעין מסוגל לבצע Look-up מהיר ב-Socket Table ולנתב כל פקטה נכנסת ל-File Descriptor המדויק של החיבור המתאים, למרות שכולם משתמשים באותו פורט מקומי 80.

שאלה מס' 6

מצגות הקורס מציגות אסימטריה ברורה ברצף קריאות המערכת בין הלקוח (TCP-client-sender) לשרת (TCP-server-receiver) בשלבי הקמת החיבור. פרטו את רצף הקריאות של השרת לעומת הלקוח, הסבר את האסימטריה הפונקציונלית ביניהם (Active Open לעומת Passive Open), והסבירו מדוע מרגע מעבר החיבור למצב ESTABLISHED האסימטריה הזו נעלמת לחלוטין ברמת הפרוטוקול.

אסימטריה מבנית בשלבי האתחול של Sockets API

בשלב האתחול, קיים שוני ארכיטקטוני מהותי ברצף הפעולות:

- **רצף השרת (Passive Open):** השרת קורא ל-`socket()` ליצירת האובייקט, מבצע `bind()` כדי לקבע את הסוקט לשיטת מיעון מקומית (Port & Local IP), קורא ל-`listen()` כדי להנחות את הגרעין להקצות תורים ולקבל בקשות רשת, ולבסוף קורא ל-`accept()` כדי להמתין לחיבורים. זהו מודל סביל (Passive) הממתין לאירועים חיצוניים.
- **רצף הלקוח (Active Open):** הלקוח קורא ל-`socket()` ולאחר מכן קורא ישירות ל-`connect()` תוך אספקת שיטת המיעון של היעד (Port & Remote IP). זהו מהלך אקטיבי (Active) היוזם את פקטת ה-SYN הראשונית. הלקוח לרוב מדלג על קריאה מפורשת ל-`bind()`, והגרעין מקצה עבורו פורט חולף (Ephemeral Port) באופן אוטומטי.

מרגע שתהליך ה-Handshake הושלם והסוקטים בשני הצדדים עברו למצב ESTABLISHED, האסימטריה נעלמת לחלוטין.

פרוטוקול TCP בבסיסו הוא פרוטוקול סימטרי לחלוטין מסוג Full-Duplex. שני הצדדים מחזיקים כעת בבאפרים עצמאיים לשליחה ולקבלה, שניהם מנהלים מנגנוני בקרת זרימה (Flow Control) ובקרת גודש (Congestion Control) נפרדים, ושני הקצוות רשאים לבצע קריאות `read()` ו-`write()` סימולטניות או ליזום את סגירת החיבור (Graceful Shutdown) באופן עצמאי.

שאלה מס' 7

הבחנה חדשה בין מנגנון בקרת זרימה (Flow Control) למנגנון בקרת גודש (Congestion Control) חיונית להבנת ביצועי רשתות. א. הגדירו במדויק את המטרה ההנדסית של כל מנגנון. ב. הסבירו אילו משאבים מערכת הפעלה ורשת מנסה כל אחד מהם להציל, וכיצד משתקפים מנגנונים אלו בחישוב גודל חלון השליחה האפקטיבי של ה-TCP.

הבחנה בין בקרת זרימה לבקרת גודש היא אכן יסוד בהבנת התנהגות ה-Transport Layer. למרות ששניהם משפיעים על קצב השליחה, הם מטפלים בבעיות שונות לחלוטין.

א. הגדרה הנדסית של המנגנונים

1. בקרת זרימה (Flow Control):

- **המטרה:** למנוע מצב שבו השולח (Sender) "מציף" את המקבל (Receiver) בנתונים מהר יותר ממה שהאפליקציה בצד המקבל מסוגלת לעבד ולקרוא מה-Receive Buffer.

- **המוקד:** יחס של נקודה-לנקודה (End-to-End) בין שולח ספציפי למקבל ספציפי.

2. בקרת גודש (Congestion Control):

- **המטרה:** למנוע מצב שבו השולח "מציף" את הרשת (הנתבים והמתגים שבדרך) בנתונים מהר יותר ממה שהקיבולת של עורקי התקשורת מאפשרת. מטרתו היא למנוע אובדן חבילות כתוצאה מ-Queue Overflow בנתבים, מה שמוביל לשידור חוזר (Retransmission) ומחמיר את העומס.
- **המוקד:** יחס של רשת כוללת; השולח מנסה "להתחשב" בתשתיות המשותפות לכלל המשתמשים.

ב. משאבים מוגנים וחשוב גודל החלון האפקטיבי

1. המשאבים המוגנים

- **בבקרת זרימה:** המשאב המוגן הוא זיכרון ה-Buffer בצד המקבל. אם המקבל לא קורא את הנתונים מספיק מהר, ה-Buffer יתמלא והנתונים הבאים יאבדו (Dropped) בגלל חוסר מקום, לא בגלל עומס ברשת.
- **בבקרת גודש:** המשאב המוגן הוא רוחב הפס של הלינקים ברשת וזמן ההמתנה (Queues) בנתבים. מניעת גודש מונעת את תופעת ה-"Congestion Collapse", שבה הרשת עסוקה בשידורים חוזרים במקום בהעברת נתונים חדשים.

2. חישוב גודל החלון האפקטיבי

השולח ב-TCP מנהל שני משתני חלון נפרדים, והחלון האפקטיבי (Effective Window) שבו הוא משתמש לקביעת קצב השליחה הוא ה-מינימום ביניהם:

$$\text{Effective Window} = \min(RWND, CWND)$$

- **(RWND (Receive Window):** זהו המשתנה של בקרת הזרימה. הלקוח מעדכן את השולח באמצעות שדה ה-Window Size ב-TCP Header, בהתבסס על המקום הפנוי ב-Receive Buffer שלו.
- **(CWND (Congestion Window):** זהו המשתנה של בקרת הגודש. זהו פרמטר פנימי ששמור ב-Kernel של השולח בלבד (לא נשלח במפורש בכותרת ה-TCP). הוא גדל (למשל, ב-Slow Start או Congestion Avoidance) ככל שהשולח מזהה שהרשת פנויה, וקטן דרסטית כשהשולח מזהה סימני גודש (אובדן חבילות או Timeout).

סיכום אופרטיבי:

- אם המקבל איטי – **RWND** יקטן ויגביל את השולח.
- אם הרשת עמוסה – **CWND** יקטן ויגביל את השולח.
- השולח לעולם לא ישלח יותר ממה ששני המנגנונים מתירים לו יחד.

שאלה מס' 8

הסבירו כיצד ייתכן שממשק BSD Sockets (ממשק ברקלי) ממומש על גבי ליבות מערכות הפעלה שונות לחלוטין (כגון Windows ו-Linux), מבלי ליצור שגיאות בתקשורת ביניהן.

ממשקי תכנות (API) ותאימות

ממשק הסוקטים הוא חוזה מקומי בלבד בין האפליקציה למערכת ההפעלה המארחת. ברגע שהנתונים 'נכנסים' לליבה (Kernel), התקשורת מתנהלת על פי אפיון הפרוטוקול הבינלאומי (למשל RFC 9293). לכן, הממשק (BSD או Winsock) הוא רק "שפה" מקומית להעברת נתונים לקרנל. הפרוטוקול שעל הכבל הוא אוניברסלי, ולכן אין משמעות לממשק התכנות של צד אחד עבור צד אחר.

שאלה מס' 9

לצורך מענה כל השאלה הבאה, עליכם לקרוא על המושג PMTUD [= Path MTU Discovery].

א. תארו את התלות שבין ה-MSS (Maximum Segment Size) לבין ה-Path MTU Discovery (PMTUD).

ב. מה קורה כאשר ה-PMTUD נכשל?

ה-MSS מגדיר את גודל הנתונים המקסימלי לכל סגמנט. ה-PMTUD מנסה לגלות מהו ה-MTU המינימלי לאורך כל הנתיב כדי למנוע פרגמנטציה ב-IP. אם ה-PMTUD נכשל **ולמשל בגלל חסימת ICMP בראטרים - "Black Hole"** ה-TCP ינסה לשלוח סגמנטים גדולים מדי, שיימותו בדרך. התוצאה היא "חיבור שבור" - נפתח חיבור, אך הנתונים לא עוברים. הפתרון המודרני הוא **MSS Clamping** או שימוש ב-Packetization Layer PMTUD.

שאלה מס' 10

בהקשר לשדרוגים השונים שנעשו למימוש של TCP במהלך השנים, אחד השדרוגים נקרא: Zero-Copy Networking

עליכם :

- לקרוא על המושג הזה
- להסביר אותו
- להתייחס לקשר שלו למניעת העתקות זיכרון בין ה- User-space לבין ה- Kernel-space בעת שליחת סגמנט TCP.

במימוש קלאסי, נתונים מועתקים מה- Buffer של האפליקציה ל- Kernel Buffer, ומשם ל- DMA Buffer של כרטיס הרשת.
Zero-Copy מנסה לבטל זאת. **על ידי שימוש בטכניקות כמו sendfile() או DMA מתקדם**, הקרנל "מצביע" ישירות על זיכרון האפליקציה **(בשימוש ב- Page pinning)**.
זה קריטי לביצועים גבוהים כי העתקת זיכרון היא פעולה יקרה הצורכת מחזורי מעבד יקרים מליבות המערכת.

שאלה מס' 11

תפתור באופן מלא את השאלה הבאה שאני מעלה לך.
ותצרף הסברים מאוד מפורטים ונרחבים לכל תשובה.

נתון :

- TCP בצד השרת מקצה סוקט.
- בעקבות קבלת SIN segment.
- ה- ISN של הסוקט הזה נקבע, אקראית, להיות 2000000.
- גודל ה- send buffer המוקצה לסוקט שירות הוא 256 KB.
- גודל ה- receive buffer המוקצה לסוקט שירות הוא 256 KB.

TCP בצד הלקוח :

- מקצה לסוקט send buffer בגודל 128 KB.
- ה- ISN של הסוקט הזה נקבע, אקראית, להיות 1000000.

- האם אכן אפשרי כי גודלו של ה- send buffer בצד הלקוח יהיה שונה מגודלו של ה- receive buffer בצד השרת?
- האם אכן אפשרי כי גודלו של ה- send buffer בצד הלקוח יהיה שונה מגודלו של ה- send buffer בצד השרת?

ג. על סמך הנתונים המובאים, מהם הערכים של שדות ה- TCP header של ה- SYN segment.

ערכים שניתן בוודאות לקבוע את ערכם.
ד. לגבי אלו שדות של הכותרת של ה-SYN segment לא ניתן לקבוע בוודאות את ערכם על סמך הנתונים המובאים?

ה. האם ה-SYN segment הראשון שנשלח יגיע בוודאות ל-TCP בצד השרת?
ו. האם ה-SYN segment הראשון שנשלח יגיע בוודאות תקין ל-TCP בצד השרת?
ז. האם TCP בצד השרת ישלח תמיד SYN-ACK segment בעקבות קבלת SYN segment תקין?

ח. על סמך הנתונים המובאים, מהם הערכים של שדות ה-TCP header של ה-SYN-ACK segment.
ערכים שניתן לקבוע את ערכם.
ט. לגבי אלו שדות של הכותרת של ה-SYN-ACK segment לא ניתן לקבוע בוודאות את ערכם על סמך הנתונים המובאים?

א.
כן, זה בהחלט אפשרי ואף נפוץ.
• **הסבר:** גודל ה-buffer (חוצץ) הוא הגדרה מקומית לכל צד בחיבור. ה-send buffer בלקוח משמש לשמירת נתונים שהאפליקציה בלקוח שלחה וטרם אושרו על ידי השרת, בעוד ה-receive buffer בשרת משמש לקליטת נתונים מהרשת לפני שהאפליקציה בשרת קוראת אותם. אין ביניהם שום תלות פרוטוקולית; לכל צד יש את המשאבים והצרכים שלו (למשל, שרת עשוי להקצות זיכרון רב יותר כי הוא משרת לקוחות רבים במקביל).

ב.
כן, זה אפשרי בהחלט.
• **הסבר:** בדומה לסעיף א', הקצאת ה-buffer היא החלטה של מערכת ההפעלה המקומית (OS) בהתאם למשאבי המערכת, סוג האפליקציה וקונפיגורציית הזיכרון. אין שום דרישה ב-TCP שגודל חוצצי השידור בשני הצדדים יהיה זהה.

ג.
על סמך הנתונים שסיפקת:
• **Sequence Number:** הערך הוא 1,000,000 (זהו ה-ISN שנקבע ללקוח).
• **Flags:** שדה ה-SYN חייב להיות דלוק (1), ושאר הדגלים (כמו ACK, FIN, RST) צריכים להיות כבויים (0).
• **Window Size:** ניתן לקבוע את הערך אם היינו יודעים את גודל ה-Receive Buffer של הלקוח (הערך בשדה ה-Window ב-TCP מציין את גודל החלון הזמין לקבלה).
הערה: מאחר שנתון רק ה-send buffer של הלקוח, לא ניתן לקבוע את ה-Receive Window בוודאות, אלא אם נניח שהם זהים (מה שלא תמיד נכון).

- **Source Port**: הלקוח בוחר פורט אקראי (Ephemeral port) בדרך כלל בטווח גבוה.
- **Destination Port**: לא צוין לאיזה פורט השרת מאזין (למשל 80 או 443).
- **Acknowledgment Number**: ב-SYN הראשון הוא תמיד 0 (מכיוון שאין עדיין נתונים לקבל מהשרת), אך מבחינה פרוטוקולית השדה קיים וערכו חסר משמעות עד שדגל ה-ACK דלוק.
- **Checksum**: תלוי בתוכן ה-IP ובתשדורת בפועל.
- **Options**: לא ניתן לדעת אילו אופציות הלקוח יבקש (למשל: MSS, SACK, Window, Scaling).

ה.

לא.

- **הסבר**: רשתות IP הן "Best Effort". חבילות עלולות ללכת לאיבוד בדרך עקב עומס בנתבים (Congestion), תקלות פיזיות בכבלים, או חסימות מצד Firewalls בדרך. TCP מתמודד עם זה באמצעות מנגנון ה-Retransmission.

ו.

לא.

- **הסבר**: מעבר לאובדן חבילות, חבילה יכולה להגיע פגומה (Corrupted) עקב הפרעות אלקטרומגנטיות. במקרה כזה, ה-Checksum בשרת לא יתאים, והחבילה תיזרק (dropped) על ידי השרת והלקוח יצטרך לשלוח אותה שוב.

ז.

לא.

- **הסבר**: השרת לא ישלח SYN-ACK אם:
 1. השרת אינו מאזין בפורט המבוקש (ישלח RST).
 2. יש מדיניות אבטחה (Firewall) שחוסמת את החיבור.
 3. ה-Backlog (תור החיבורים הממתינים) של השרת מלא (השרת עמוס מאוד).

ח.

- **Sequence Number**: הערך הוא 2,000,000 (ה-ISN של השרת).
- **Acknowledgment Number**: הערך יהיה 1,000,001 (ה-ISN של הלקוח + 1, כיוון ש-SYN צורך מספר רצף אחד).
- **Flags**: דגלי SYN ו-ACK שניהם דלוקים (1).

ט.

- **Window Size** : תלוי ב-Receive Buffer של השרת (שאמנם נתון כ-256KB, אך ה-Window ב-TCP מחושב לעיתים עם Window Scaling, ולא ניתן לדעת מהו ללא ידיעת ה-Options).
- **Source/Destination Ports** : אלו יהיו היפוך של הפורטים מההודעה המקורית.
- **Options** : לא ניתן לדעת אילו אופציות השרת יסכים להפעיל (כמו SACK או Time-stamps).

שאלה מס' 12

מה המשמעות של cumulative ACK?

המונח **Cumulative ACK** (אישור מצטבר) הוא אחד ממנגנוני היסוד של פרוטוקול TCP, שנועד להבטיח אמינות בתקשורת נתונים בצורה יעילה.

להלן הסבר מפורט על משמעותו ואופן פעולתו :

המשמעות והקונספט

בעולם של TCP, כאשר צד אחד (השולח) שולח רצף של בתים (Bytes) לצד השני (הקבל), הוא לא מצפה לקבל אישור נפרד עבור כל בית בודד או עבור כל חבילה (Segment) בנפרד.

Cumulative ACK משמעו: כאשר ה-TCP בשרת שולח אישור (ACK) ללקוח, הערך שנמצא בשדה ה-Acknowledgment Number מייצג את **מספר הרצף (Sequence Number) של הבייט הבא שהשרת מצפה לקבל**.

- **במילים פשוטות**: אם השרת קיבל את כל הבתים עד בייט מספר N, ה-ACK שישלח השרת יכיל את המספר N+1.
- **המשמעות עבור השולח**: ברגע שהשולח מקבל ACK עם הערך N+1, הוא יודע בוודאות שכל הבתים שקדמו ל-N+1 התקבלו בהצלחה בצד השני.

דוגמה להמחשה

נניח שהלקוח שולח לשרת 4 חבילות נתונים :

1. חבילה 1 : מכילה בתים 1-1000.
 2. חבילה 2 : מכילה בתים 1001-2000.
 3. חבילה 3 : מכילה בתים 2001-3000.
 4. חבילה 4 : מכילה בתים 3001-4000.
- אם כל החבילות הגיעו לשרת, השרת לא חייב לשלוח 4 אישורים נפרדים. הוא יכול לשלוח **ACK אחד בלבד** עם הערך **4001**.
 - ברגע שהלקוח מקבל ACK=4001, הוא "מבין" באופן מצטבר שחבילות 1, 2, 3 ו-4 הגיעו כולן בשלום.

למה משתמשים בזה? (יתרונות)

1. **חיסכון ברוחב פס:** אין צורך לשלוח חבילת ACK עבור כל Segment קטן שנכנס. זה מפחית משמעותית את ה-Overhead (תעבורה מיותרת) ברשת.
2. **עמידות בפני אובדן אישורים:** אם אישור אחד הלך לאיבוד בדרך, ה-ACK הבא שיגיע יהיה בעל ערך גבוה יותר ויכסה גם את הנתונים שהאישור הקודם אישר. לדוגמה, אם השרת שלח ACK=1001 והוא אבד, ואז שלח ACK=2001 – הלקוח עדיין ידע שכל מה שעד 2000 הגיע, ואין צורך לשלוח מחדש.

נקודה חשובה: הסדר והחמצה

ה-Cumulative ACK עובד רק כל עוד אין "חורים" (Gaps) ברצף הנתונים. אם השרת קיבל את חבילות 1, 2 ו-4, אבל חבילה 3 הלכה לאיבוד:

- השרת ימשיך לשלוח ACK=1001 (כי הוא מחכה לבייט ה-1001, שזה תחילת חבילה 3).
- גם אם הוא קיבל את חבילה 4, הוא לא יוכל להעלות את ה-ACK מעבר ל-1001, כי הוא לא יכול "לדלג" על חבילה 3.